Phase B Technical Report

Olist E-Commerce Business Intelligence and Decision Support System

Project: Olist E-Commerce Business Intelligence
Phase: Phase B — Model Explainability, Business Dashboard, and Application Testing
Application: Streamlit
Dataset: Brazilian Olist E-Commerce Dataset

1. Executive Summary

In Phase B, I extended my Phase A work by preparing a business intelligence application that presents e-commerce data, customer segmentation, sales trends, sales forecasts, predictive results, and model explanations. I reused the processed data and trained models from Phase A rather than unnecessarily repeating the entire modelling process. I also prepared an application testing plan to evaluate the functionality and reliability of the dashboard.

2. Business Problem and Objectives

I focused on making e-commerce analysis more accessible for business decision-making. Raw datasets and separate model outputs can be difficult to interpret, so I developed a dashboard approach that brings important information together.

My objectives were to:

- Present important business indicators in one dashboard.
- Display customer segments and their characteristics.
- Visualize historical sales and forecast results.
- Provide access to model predictions and feature-importance information.
- Identify model risks and limitations.
- Test the application and document its behaviour.

3. Dataset and Data Integration

I continued using the Brazilian Olist E-Commerce Dataset from Phase A. The project included processed analytical data and supporting outputs for customer analysis, sales forecasting, model evaluation, and feature importance.

I organized these outputs so that the Streamlit application could use the relevant files for its dashboard sections. I also retained data-quality reports, model-training results, forecast-evaluation results, and prediction-error outputs as supporting evidence.

4. Data Cleaning and Quality Checks

I reused the data preparation and quality checks completed in Phase A. These checks included examining missing values, duplicate records, data types, and consistency across the integrated data.

I retained the before-and-after data-quality reports to support the documentation of data preparation. I used the processed analytical dataset for the dashboard and kept supporting quality reports for reference.

5. Exploratory Data Analysis and Feature Engineering

I reused the exploratory analysis and engineered features produced in Phase A. These steps supported the understanding of customer behaviour, sales activity, and the input variables used by the predictive model.

I used the available customer profiles, segmentation outputs, sales records, and feature-importance results to prepare the dashboard. I did not assume that a feature was important or that a customer segment had a particular business meaning without checking the actual results.

6. Predictive Modelling and Evaluation

I reused the trained model from Phase A for the prediction and model-evaluation components. I used the held-out test results, classification metrics, confusion matrix, and prediction-error analysis to examine model performance.

Final model: Tuned Random Forest Classifier

Test accuracy: 89.84%

Weighted F1-score: 0.4413 (44.13%)

I interpreted the evaluation metrics together rather than relying only on accuracy. I also considered whether the model performed differently across classes and whether its training performance suggested possible overfitting.

7. Explainable AI and Risk Analysis

I prepared feature-importance information to help explain which input features the fitted model relied on most. Feature importance describes model behaviour, but it does not prove that a feature causes a business outcome.

I considered several risks, including incorrect predictions, class imbalance, missing or inconsistent data, overfitting, and changes in future data. I recommend reviewing uncertain or consequential predictions, monitoring performance, validating input data, and evaluating precision and recall alongside other metrics.

8. Business Dashboard and Application

I prepared a Streamlit dashboard in app.py with seven main sections:

1. **Executive Overview:** Displays available business indicators and a data preview.
2. **Customer Segmentation:** Presents cluster sizes and available customer profiles.
3. **Sales Trends:** Visualizes monthly revenue when suitable date and revenue columns are available.
4. **Forecasting:** Displays available forecast results and calculates error metrics when actual and predicted values are present.
5. **Prediction:** Uses the saved model to generate predictions when the model and its required input features are available.
6. **Model Explanation:** Displays feature-importance information and describes model risks.
7. **Business Recommendations:** Presents potential actions informed by the analysis, segmentation, forecasts, and model results.

I used requirements.txt to list the Python libraries needed to run the application. I designed the dashboard to reuse saved data and models rather than train a new model every time it opens.

9. Decision Support and Business Implementation

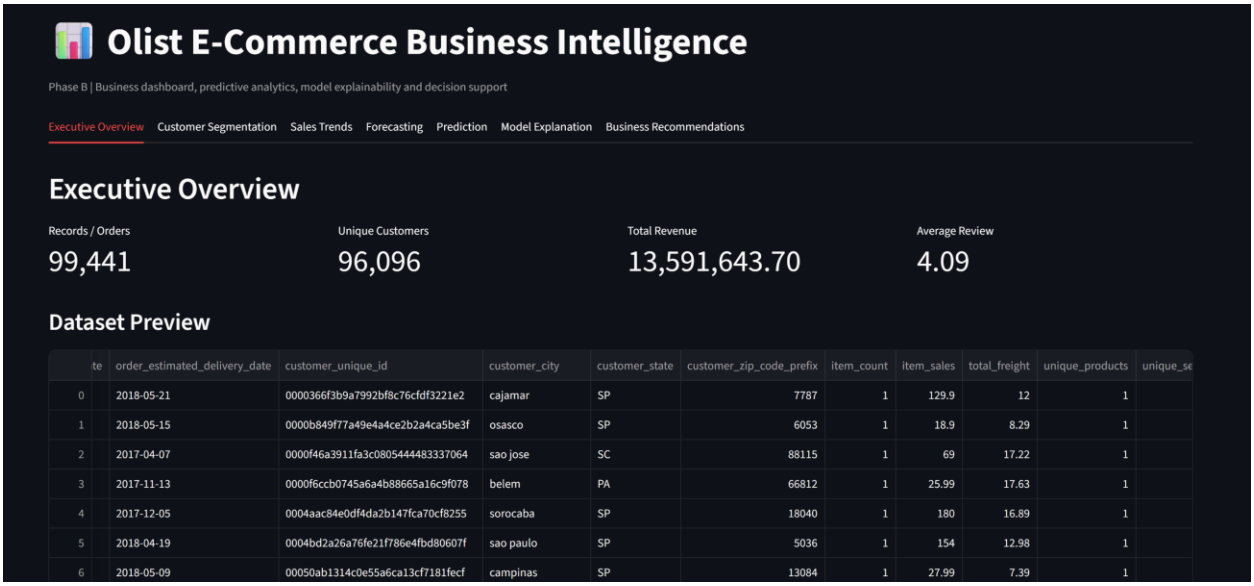
I designed the dashboard to help users review business indicators, compare customer segments, inspect sales trends, understand forecast results, and examine model predictions.

Based on the evidence available, I can use the dashboard to support customer-targeting analysis, sales planning, and the investigation of review-score patterns. I would validate each recommendation against the actual dataset and model results before using it in a business decision.

10. Application Testing

I prepared an application test file, application_test_cases.csv, to record the expected and actual behaviour of the dashboard. The test cases cover application launch, data loading, customer segmentation, sales trends, forecasting, model loading, prediction, feature importance, business recommendations, and handling missing or incompatible files.

I recorded each test's actual outcome after running the application. A test is considered successful only when the observed behaviour matches its expected result.



	te	order_estimated_delivery_date	customer_unique_id	customer_city	customer_state	customer_zip_code_prefix	item_count	item_sales	total_freight	unique_products	unique_se
0		2018-05-21	0000366f3b9a7992bf8c76cfdf3221e2	cajamar	SP	7787	1	129.9	12	1	
1		2018-05-15	0000b849f77a49e4a4ce2b2a4ca5be3f	osasco	SP	6053	1	18.9	8.29	1	
2		2017-04-07	0000f46a3911fa3c0805444483337064	sao jose	SC	88115	1	69	17.22	1	
3		2017-11-13	0000f6ccb0745a6a4b88665a16c9f078	belem	PA	66812	1	25.99	17.63	1	
4		2017-12-05	0004aac84e0df4da2b147fca70cf8255	sorocaba	SP	18040	1	180	16.89	1	
5		2018-04-19	0004bd2a26a76fe21f786e4fbd80607f	sao paulo	SP	5036	1	154	12.98	1	
6		2018-05-09	00050ab1314c0e55a6ca13cf7181fecf	campinas	SP	13084	1	27.99	7.39	1	

Figure 1overview

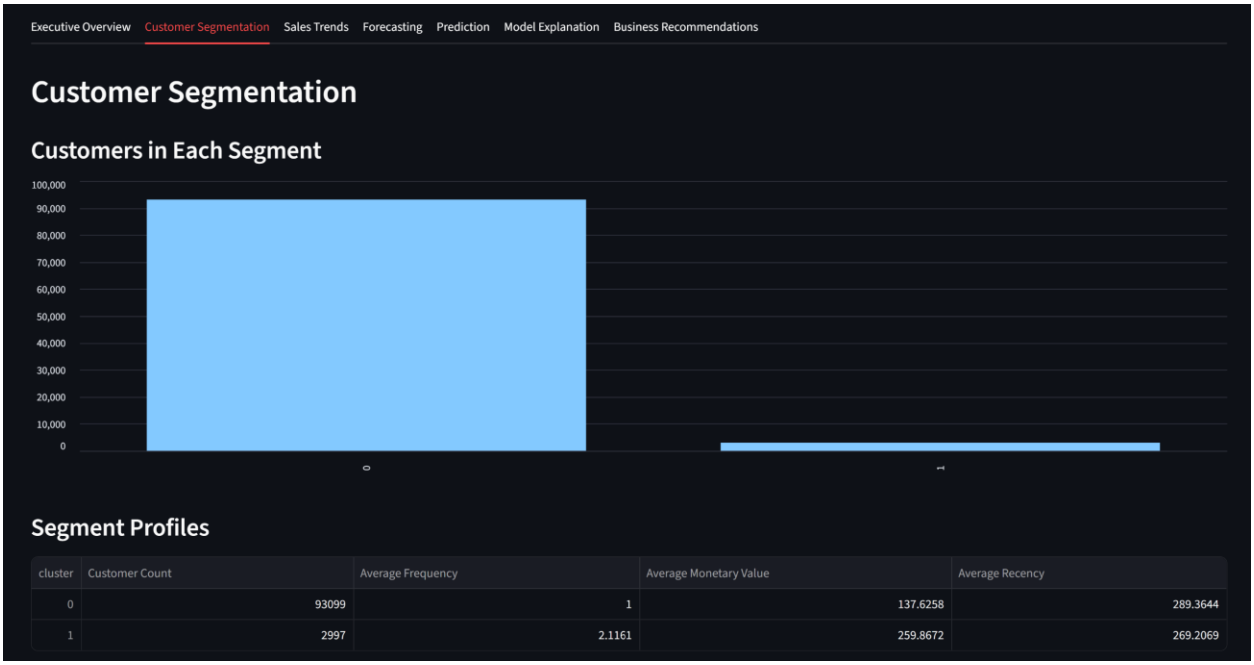


Figure 2 customer segmentation

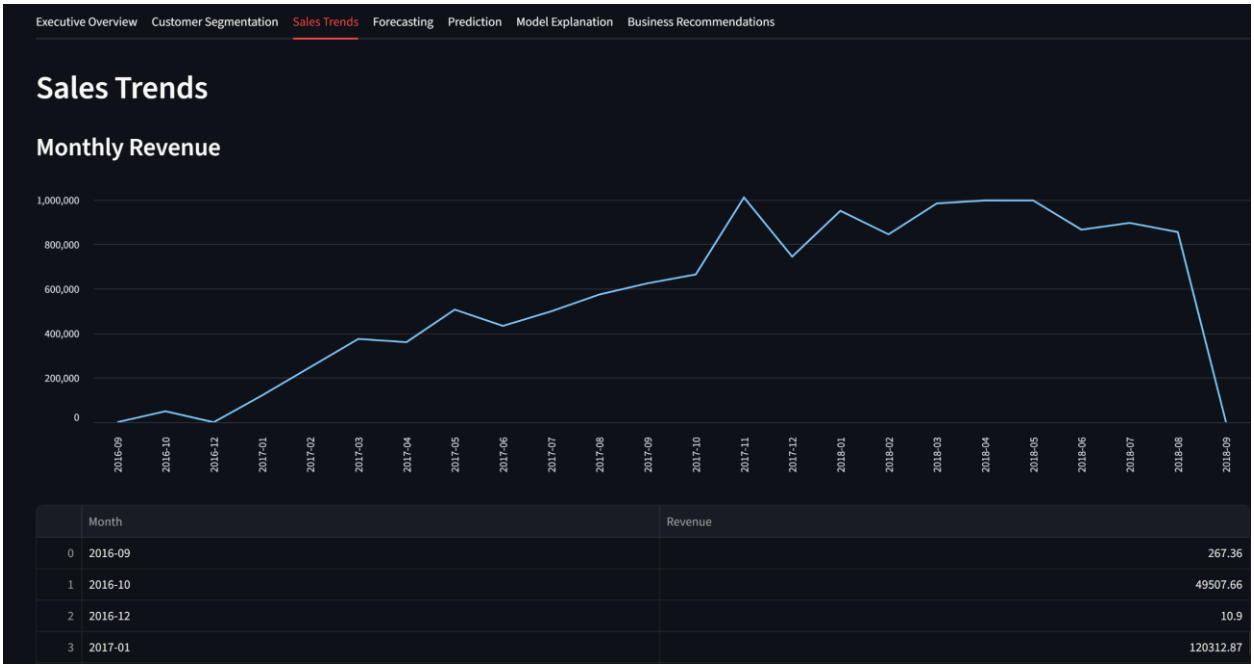


Figure 3 sales trends



Figure 4 sales forcasting



Figure 5: model explanation and risk analysis

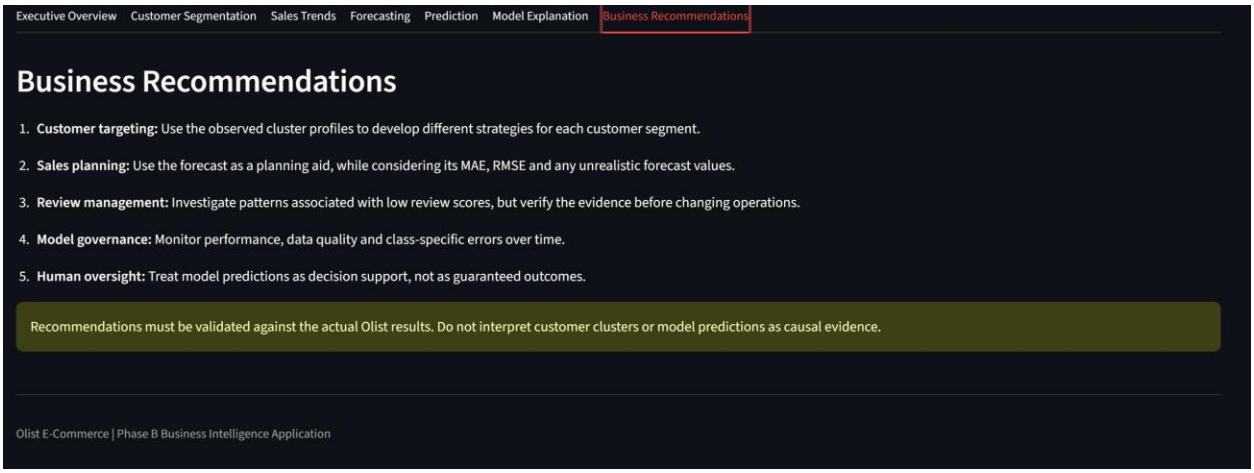


Figure 6 business recomdationbs

11. Limitations and Risks

The dashboard depends on the availability and correct structure of the processed CSV files and saved model. Automatic column detection may not recognize every dataset schema, so some column mappings may require adjustment.

Forecast results are subject to prediction error and should not be treated as guaranteed future sales. Customer clusters depend on the selected features and clustering method. Predictive outputs may contain errors and should be interpreted with appropriate human oversight.

I also recognize that a dashboard displaying a prediction does not, by itself, prove that the prediction is reliable or that a business recommendation will be effective.

12. Conclusion and Future Improvements

In Phase B, I prepared the Streamlit dashboard structure, reused the outputs from Phase A, and organized model-explanation and application-testing activities. The application brings the main analytical components into a single interface and provides a foundation for business decision support.

For future improvements, I would refine the dashboard's input validation, add more detailed model explanations, improve error monitoring, strengthen automated testing, and evaluate the system using additional real-world data. I would also improve deployment security and monitor model performance after deployment.

-